



BITS Pilani
Hyderabad Campus

Birla Institute of Technology and Science, Pilani Hyderabad Campus
2nd semester 2026, CS F407 (Artificial Intelligence),
(Evaluative & Take Home) Project Assignment - II (Total – 60 Marks, 20% Weight)

Instructor In-Charge: Prof. Sk Aziz Ali (email: skaziz.ali@hyderabad.bits-pilani.ac.in)

Assignment Start Date: 15/04/2026 at 10:00 AM

Submission Deadline: 10/05/2026 at 05:00 PM (**No Extension! No Partial Marking within Exercise!**)

General Instructions:

- Create a **PRIVATE (NOT Public) GitHub** repository named after your group (e.g., CSF407_2025_ID1, CSF407_2025_ID2,...). The GitHub repository must contain:
 - o A folder named 'src' to contain all source code of Python files *.py, etc.
 - o A file named README.md to document your project execution and results
 - o A config.yml to build a local **conda** environment of the same name as your group
- Additionally, you can upload your solutions as a Jupyter Notebook file (.ipynb) and/or additional Python files as per the instructions set in the questions, results as images/simulation videos, and any other results/files generated while solving the project assignment inside the 'src' folder.
- **ONLY ONE member of a Group should submit** the solution by emailing IC with the email subject [CSF407_2026_ID_Assignment-II] and **ONLY** one zip file named CSF407_2026_ID_Assignment-II.zip . GroupLeaderID is the **ID** Number of the Group Leader. If multiple solutions are submitted, one will be randomly selected for evaluation. (**Note: **[5 Marks]** for following all the instructions perfectly)

Exercise-1: Design and implement an AI system that uses an artificial neural network-based Machine Learning (ML) model integrated inside an *aerial drone platform* to autonomously identify the safest and closest landing zone for a package drop from a single downward-facing camera frame with respect to a *given target* location. Using semantic segmentation, monocular depth estimation, and a graph-based semantic reasoning engine, the AI system must search and select the safe region with the *optimal score* based on the cost function. **[55 Marks]**



Street View



Aerial View from Drone

Figure 1. Difference Aerial and Drone View



→(A) Dataset to use:

You must only use the TU Graz Semantic Drone Dataset available [here](#), by downloading only *semantics_drone_dataset_semantics_v1.1* zip file. The details of the dataset – e.g., number of classes, image resolution, and safe zone class ID masks – are available on the [dataset's homepage](#). The figure above explains how a street view of a scene appears from a drone.

→(B) Artificial Neural Network (ANN) Model for the AI System:

1. **ANN training** on the Graz image dataset to predict a per-pixel class label (0–22) from a 256×256 RGB image. Use Cross-Entropy Loss and the Adam optimizer. Save the best checkpoint to *best_model.pth* based on validation loss. The ANN architecture must be strictly limited to four encoder layers and four decoder layers, and it must be trained only on the training images; validation can be done on the test set. This is called a semantic segmentation task (see Fig.2)..
2. **Preprocessing Before ANN training starts**, you resize (better, normalize) the raw input image to a fixed 800×600 as your main image to ensure consistent geometric distances and bounding box calculations across the dataset. (see Fig.2).
3. **During Inference using the trained ANN Model**, downscale any input image to a resolution of 256×256 and feed the ANN model for fast inference. Then upscale its output mask back to the original 800×600 without losing true scale factors. **The solution will be considered invalid if these scaling rules are not followed** (see Fig.2).
4. **Hardware for training ANN model:** The free GPUs provided by Kaggle (more than sufficient ~20 Hours per week on T4 type of GPUs), Google Colab for training (~15-20 hours per week on T4 type of GPUs), or your PC if a GPU is available.

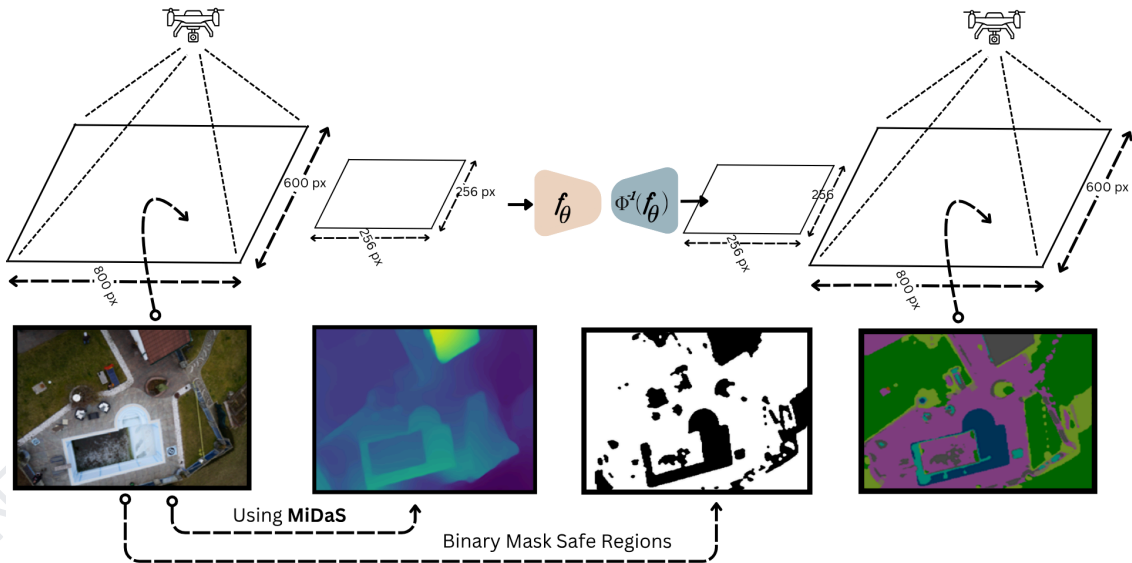


Figure 2. Overview of the Data and ANN of the AI system

→(C) Depth Estimation using the MiDaS Model for the AI System:

1. **Use Intel MiDaS (MiDaS_small via torch.hub)** to produce a relative depth map image. Normalize the depth (in this case, the height) to uint8 [0–255] and validate using colormap. Bright pixels = close to the drone; dark = far (Fig.2). Used to compute terrain slope under each candidate box.



→(D) Building Binary Mark and Safe Zone Searching for the AI System:

1. **Create a binary safety mask** from the semantic segmentation map by marking pixels of classes 1, 3, and 4 as white and all others as black (see **Fig.2**).
2. **An optimized search algorithm** can be used to find valid placement regions where a rotatable bounding box fits entirely within image boundaries. Think about how to select only interior sample points that fall on white pixels, and then how to reject such interior points from the safety zone where the mean slope or pixel gradient of the depth images beneath those points stays below 0.15.
3. One should **optimize** the safe-region search so that it returns the safe region result quickly.
NOTE: the **target location** need not be a safe place; it can be any pixel coordinate in the input test-time image.

→(E) Knowledge-Graph for Semantic Reasoning based on Input Mission Configuration:

1. At runtime/inference time, the AI system must read the **mission_config.json** file shown on the right and inspect the nature of the drop-in package block as the drone's payload. For each property within that block, apply the following mapping rule.
2. If a property is present and evaluates to true, activate the corresponding source node in the knowledge graph as specified in the table below.
3. Implement a **3-layer spreading activation graph** that computes a penalty score for each terrain type based on the active traits of the package being delivered. All penalty scores must be derived **solely from edge-weight accumulation** across the graph. No **if/else** chains and no hardcoded penalty values are permitted anywhere in the pipeline.
 - a. Design a directed weighted graph with the following three fixed layers:
 - **Layer 1 — Source Nodes:** Fragile, Valuable, Biohazard, Heavy
 - **Layer 2 — Property Nodes:** Hard, Wet, Slippery, Dirty, Visible, Contaminated, Soft, Unstable
 - **Layer 3 — Terrain Nodes:** Pavement, Grass, Dirt**Edges run Layer 1 → Layer 2 and Layer 2 → Layer 3 only. You are responsible for designing all edge weights meaningfully.**
4. Therefore, there must be a source file named **semantic_brain.py** to identify the dominant terrain within a candidate bounding box by sampling interior points, finding the most frequent segmentation class (e.g., 1,3,4), and mapping it to their corresponding terrain type (e.g., "Pavement", "Grass", "Dirt") table entries, and calculating the semantic penalty score.

```
{  
  "mission_id": "OP-DELTA-9"  
  "package": {  
    "type": "medical_vials",  
    "heavy": true,  
    "fragile": false,  
    "valuable": false,  
    "biohazard": false  
  }  
}
```

Example — suppose active nodes [Fragile, Valuable] – grass should receive the lower penalty score than other safe landing terrains (pavement and dirt) for soft-landing. For instance, with active nodes such as [Fragile, Valuable], terrains like grass should incur a lower penalty score for a soft landing compared to other safe landing areas (pavement and dirt).



→(F) **Optimize Cost Function and Find Optimal Target Landing Location:**

1. Once valid candidate regions have been identified, rank them by computing a scalar cost for each candidate. The optimal placement or target location is the candidate with the **lowest cost**. The cost function integrates three components:
 - 1.1. proximity to the user-specified target (i.e., **B**)
 - 1.2. Roughness of the terrain class type from the depth map, and (i.e., **C**)
 - 1.3. semantic suitability from the knowledge graph. (i.e., **D, E**)

Cost function with definition of three terms:

$$\text{Cost} = 0.4 \times \text{distance} + 0.2 \times \text{roughness} + 0.4 \times \text{semantic_penalty}$$

1. Distance

The normalised Euclidean pixel distance from the candidate region's centre $C = (x_c, y_c)$ to the user-specified target coordinate $t = (x_t, y_t)$:

$$\text{distance} = \frac{\sqrt{(x_c - x_t)^2 + (y_c - y_t)^2}}{1000}$$

2. Roughness

Compute the mean Sobel gradient magnitude from the MiDaS depth map across all interior sample points of the candidate bounding box. Any candidate whose raw mean Sobel magnitude exceeds the threshold **MAX_SLOPE = 0.15** must be **strictly discarded** before cost evaluation — it is not a valid candidate regardless of its other scores. For all remaining candidates, normalise as:

$$\text{roughness} = \frac{\text{mean Sobel magnitude}}{0.15}$$

3. Semantic Penalty

Retrieve the terrain penalty from the knowledge graph using the terrain class identified by **semantic_brain.py**:

$$\text{semantic_penalty} = \text{think}(\text{active_nodes})[\text{terrain_class}]$$

This value is in the range $[0.0, 1.0]$ as produced by the capped graph output defined in the knowledge graph assignment.

Draw a green-oriented bounding box at the lowest-cost position on the test image.



→ (G) AI System Implementation and Program Architecture:

Your submission must strictly adhere to the following modular structure for the files and implementation, and not use monolithic scripts in a single file.

1. **config.py**: Centralized configuration for all constants, including image sizes, 22-class semantic labels, cost function weights (distance, roughness, semantic), device selection, and safe landing classes. This means all parameters are separate variables.
2. **dataset.py**: Implements the GrazDataset class to load aerial images and their corresponding RGB annotation masks. Handles preprocessing, converting RGB masks to integer class IDs, and transforming images into Channel x Height x Width tensors for ANN model ingestion.
2. **model.py**: Defines the core Artificial Neural Network (CNN) architecture used for semantic segmentation.
3. **train.py**: Orchestrates the model training loop over defined epochs using Cross-Entropy loss and the Adam optimizer. Evaluates performance and automatically saves the optimal model weights to *best_model.pth* file. You need to provide your best checkpoint file.
4. **geometry.py**:
 - Generates a binary safe-zone mask based on geometrically safe classes.
 - Executes a rotational grid search across the safe mask to find a valid candidate landing box matching the package footprint.
 - Performs slope and roughness validation using the generated depth map.
 - Integrates the geometric data with the semantic penalty to calculate and sort the final normalized cost score.
5. **knowledge_graph.py**: This file module implements the semantic reasoning engine for terrain type's safety estimation.
 - Implements a 3-layer Spreading Activation graph architecture to compute terrain penalty scores. No conditional *if/else* chains are permitted anywhere in the penalty computation pipeline.
 - Defines two edge dictionaries — [SOURCE_TO_PROPERTY_EDGES](#) and [PROPERTY_TO_TERRAIN_EDGES](#) — which encode all penalty propagation rules as weighted graph connections.
 - Implements the [think\(\)](#) function, which accumulates penalty weights layer by layer for each terrain node based on the provided active source nodes, with all final scores capped at 1.0.
6. **semantic_brain.py**: Extracts interior sample points from each candidate bounding box in the original image and classifies the dominant terrain type using the segmentation output. The resulting terrain label is passed to the knowledge graph as the lookup key for retrieving the penalty score.
7. **semantic_brain.py**: Parses [mission_config.json](#) at runtime to extract the active package traits and environmental conditions. Returns the identified traits as a list of active source node names for consumption by the knowledge graph.
8. **utils.py**: for different types of utility functions – e.g., semantic color mapping, creating mask-to-rgb() or vice-versa.
9. **main.py**: The entry point and orchestration layer for the full ANN system.



- Accepts interactive CLI inputs for the target image path and the user-specified target delivery pixel coordinates.
- Loads all static configuration parameters and initializes the pre-trained neural network models.
- Executes the sequential inference pipeline in the following order: image segmentation → depth estimation → candidate region generation → semantic graph evaluation → cost ranking.
- Renders and saves the final output image, overlaying the optimal placement bounding box, the target coordinate marker, and a text breakdown of the individual and total costs for the selected candidate.

Some standard static files will also be part of the project, such as best_model.pth and requirements.txt/yml, which list all packages necessary to build a conda environment, etc.

Sample Test Case Scenario To Illustrate Input/Output

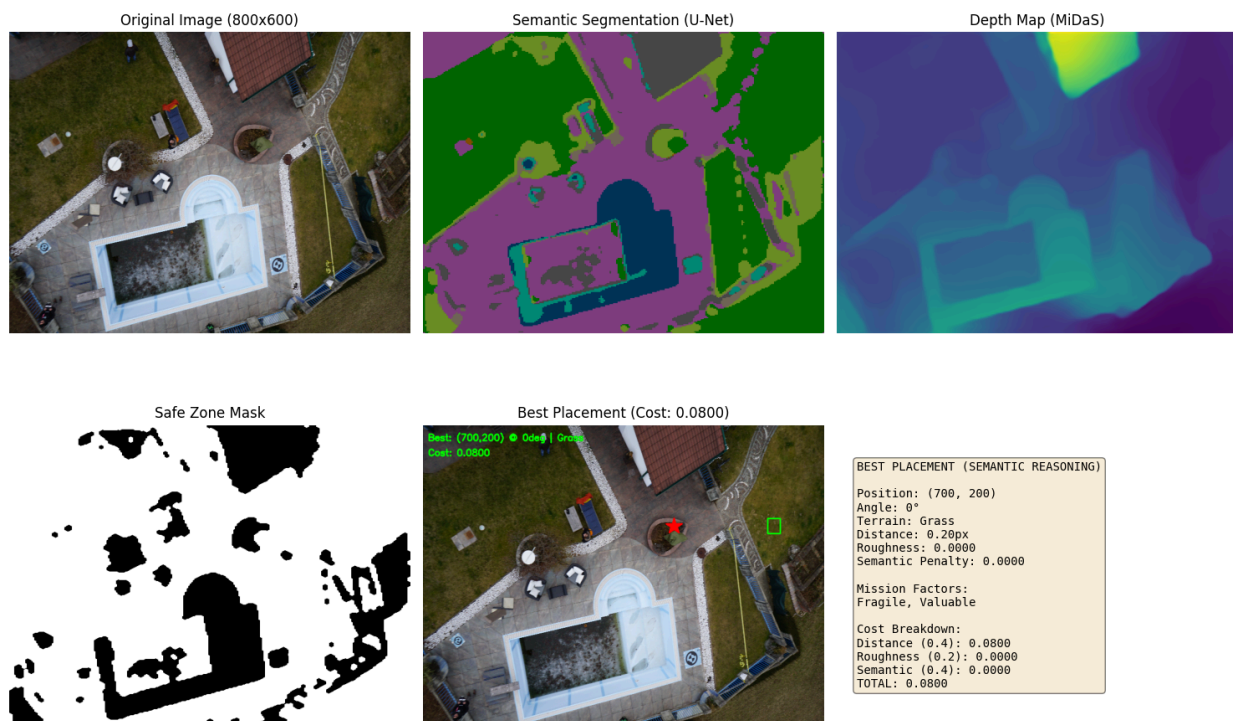
Input: User enters the image filepath followed by the target co-ordinates ($0 \leq x \leq 800$, $0 \leq y \leq 600$) of the red star-marked pixel.

Expected Output

output.jpg — input image with best placement bounding box (green) and display target coordinate

output_analysis.jpg — 6-panel dashboard: original, segmentation, depth map, safe mask, best placement, cost breakdown panel

Example-1





Example-2

